



## CHAPTER 14

# Terraform Cloud & Enterprise

Collaborative infrastructure management, remote state storage, and policy enforcement with Sentinel.

*Remote Execution · State Management · RBAC · Sentinel Policies*

# What is Terraform Cloud?

A managed service by HashiCorp for team-based infrastructure management.



## Remote Execution

Runs Terraform plans and applies in HashiCorp-managed cloud runners.



## State Management

Centralized, versioned state storage — no more local .tfstate files.



## Team Collaboration

Role-based access control (RBAC) and run notifications.



## Policy Enforcement

Sentinel policies block non-compliant infrastructure before it ships.

*Terraform Cloud simplifies team-based infrastructure management.*

### Talking point

Ask learners where they currently store Terraform state today — most will say “locally” or “S3.” Use that to set up the contrast on the next slide.

# Terraform Cloud vs. Self-Hosted

| Feature                | Terraform Cloud ☁️  | Self-Hosted Terraform 🏠 |
|------------------------|---------------------|-------------------------|
| State Management       | Built-in (SaaS)     | Manual (e.g., S3, Git)  |
| Execution Environments | Cloud Runners       | Local / CI Pipeline     |
| Security & Compliance  | Sentinel Policies   | Manual Implementation   |
| Collaboration          | RBAC, Notifications | GitOps, CI/CD Pipelines |



Terraform Cloud simplifies team-based infrastructure management — trading manual setup for a managed, governed workflow.

# Getting Started with Terraform Cloud

By default, Terraform stores state locally (terraform.tfstate). Terraform Cloud provides a centralized backend instead.

1

### Create a Terraform Cloud Account

Sign up at [app.terraform.io](https://app.terraform.io)

2

### Create a New Organization

Set up the org that will own your workspaces

3

### Create a Workspace

Provision a workspace for managing infrastructure

# Configure the Cloud Backend

## Step 2 — main.tf

```
terraform {  
  cloud {  
    organization = "my-terraform-org"  
  
    workspaces {  
      name = "my-terraform-workspace"  
    }  
  }  
}
```

## Step 3 — Initialize & run

```
$ terraform init  
  
$ terraform plan  
$ terraform apply
```



Terraform authenticates with Terraform Cloud and stores state remotely.



Runs can also be triggered directly from the Terraform Cloud UI — not just the CLI.

## 14.3 Policy Enforcement

# Using Sentinel for Policy Enforcement

Sentinel is a policy-as-code framework that enforces security rules before Terraform applies changes.

- Governance Guardrails
- Runs Before Apply
- Blocks Non-Compliant Plans



# Enable & Define a Sentinel Policy

### Step 1 — Enable Sentinel

- 1 Navigate to Settings → Policy Sets
- 2 Create a new policy set
- 3 Link the policy set to your workspace



*Policy sets link one or more Sentinel policies to workspaces, so every run is checked automatically.*

### Step 2 — restrict-region.sentinel

```
import "tfplan/v2" as tfplan

allowed_regions = ["us-east-1",
                  "us-west-2"]

main = rule {
  all tfplan.resources.aws_instance
    as _, instances {
    all instances.applied
      .resource_changes as instance {
      instance.change.after.region
        in allowed_regions
      }
    }
}
```

## 14.3 Policy Enforcement

# Test the Sentinel Policy

Step 3 — Run a policy check with terraform apply



### Region: us-east-1

Compliant — within allowed\_regions

```
$ terraform apply
```

```
Sentinel Policy Check: PASSED  
Applying: aws_instance.web  
Apply complete!
```



### Region: eu-central-1

Non-compliant — outside allowed\_regions

```
$ terraform apply
```

```
Policy Violation:  
Resource 'aws_instance.web' is in  
an unauthorized region  
'eu-central-1'.
```

*Ensures compliance with security policies before infrastructure changes ship.*



# Set Up a Workspace & Enforce Policies

## Problem Statement

You need to store Terraform state remotely and enforce security policies using Sentinel.

## Solution

1



### Create an org & workspace

Set up a Terraform Cloud organization and workspace.

2



### Use Terraform Cloud as backend

Point main.tf at the cloud block for remote state.

3



### Implement Sentinel policies

Restrict infrastructure deployments to approved regions.

## Expected Outcome



State is securely stored in Terraform Cloud



Policy checks prevent non-compliant deployments

# Key Takeaways



## Remote State & Execution

Terraform Cloud enables remote state storage and remote execution of runs.



## Sentinel Enforces Security

Sentinel enforces security policies for every Terraform deployment.



## CI/CD Integration

Terraform Cloud integrates with CI/CD pipelines for automation.